PROJECT PLAN

# Local AI for Knowledge Work

*A six-week prototype: a fully local, three-tier system that covers the everyday AI needs of a consulting firm at quality comparable to GPT/Claude — with no data leaving the building.*

## 1. What we are building (and what we are not)

The product vision is a fully local system that handles the bulk of a consulting firm's everyday AI work — summarization, document Q&A, extraction, house-style drafting — at quality comparable to GPT/Claude, with two structural advantages a hosted frontier model cannot offer: client data never leaves the firm's hardware, and there is no per-query cost.

The system is organized like a small firm: a manager that routes work, cheap specialists for known recurring tasks, and an expensive generalist for novel or hard ones.

### The three tiers

- **Router (the manager).** Classifies each incoming task and dispatches it. Known, recurring task type → cheap specialist. Novel, open-ended, or low-confidence → escalate to the generalist. The router's accuracy caps the whole system, so it is the primary design focus.
- **Specialist tier (the employees).** One base model with hot-swappable LoRA adapters — each adapter is a cheap fine-tune for a specific recurring task. Fast and cheap. This is where fine-tuning's strength is captured *inside* the system rather than competing with it.
- **Generalist tier (the senior partner).** The ensemble-plus-judge: several mid-sized open-weight models generate diverse candidates, a trained judge selects the best. Expensive, reserved for the hard tail. Over time it can also generate training data to mint new specialist adapters — described here as future work, not built in the six weeks.

### Six-week scope — the honest bar

By the end of six weeks we target a working prototype, not a finished product. Concretely:

- A working **router** that distinguishes 2–3 known task types from an 'everything else' bucket, with a confidence-based escalation rule.
- **2–3 specialist adapters** for the known task types (each a 1–2 day fine-tune).
- The **ensemble-plus-judge generalist** as the fallback path.
- A measured result: across a **basket of consulting tasks**, the routed local system matches or approaches a frontier (GPT/Claude) baseline on quality, while being cheaper than running the ensemble on everything — all running on local hardware.

*Explicitly out of scope for six weeks: broad open-ended parity with frontier models across all consulting work; the self-training-specialist loop (automated adapter creation); production hardening, security, and multi-user serving.*

## 2. Why this design, and the baseline it must beat

A fair skeptic asks two questions, and the plan answers both by building the competing approaches as explicit baselines:

- **"Why not just call GPT/Claude?"** — Because of data locality and marginal cost. The system runs entirely on-prem; client data never leaves, and there is no per-query fee. For a consulting

firm handling confidential deal data, that alone is a reason to prefer a slightly-weaker local system.

- **“Why not just fine-tune one model per task?”** — On a single narrow task, fine-tuning likely wins, and the system agrees: it *uses* fine-tuned specialists for those. The architecture earns its complexity only on **breadth** — covering many task types, including novel ones, with one system. So the POC must prove breadth-at-quality, not single-task quality.

Baselines we build and measure against: (a) frontier API (the quality bar), (b) a single fine-tuned model / fine-tune-and-route (the simplicity bar). The result is credible only if the full system is competitive with both. If the simpler fine-tune-and-route baseline wins, that is a legitimate and reportable finding.

## 3. Team, workstreams, and compute

Four people working near-full-time, coding with Claude Code. Work is split into four parallel lanes so the team maps one-to-one to workstreams, with evaluation staffed as a first-class concern rather than an afterthought.

| Workstream | Owner | Responsibility |
|---|---|---|
| Infra & Serving | Person A | Local inference (vLLM/SGLang) on baremetal, base-model serving, multi-LoRA adapter hosting, sandboxing, GPU scheduling. |
| Router & Specialists | Person B | Task classifier/router, confidence-escalation logic, training and registering 2–3 LoRA adapters, adapter registry. |
| Ensemble & Judge | Person C | Multi-model candidate generation, debate round, trained judge (teacher-labeled + human-corrected), candidate selection. |
| Eval & Benchmark | Person D | Benchmark design, frontier + fine-tune baselines, scoring harness, KPI rollup, labeling coordination. Owns the scoreboard. |

### Compute plan

- **Baremetal GPUs:** reserved for ensemble inference, the router/judge serving, and frontier-baseline + scoring runs. These must stay up, so they live here, not on Colab.
- **Colab + spare capacity:** adapter fine-tuning, which is light and parallelizable — several LoRA jobs at once. Treat Colab as a batch training surface only; session timeouts make it unsuitable for anything long-running or that must stay available.
- **Coordination, not contention:** because adapter training (Colab) and ensemble serving (baremetal) are separated, GPU collisions are unlikely — but agree a simple rule for who runs large baremetal jobs during scoring windows so eval runs aren't disrupted.

**Consequence for the schedule: parallel adapter training takes the specialist tier off the critical path. The critical path is the router and the evaluation harness — the two things that determine whether the result can be proven.**

## 4. Six-week plan

Agile cadence: weekly iterations, each ending in something measurable. A working-but-measured thin slice exists by end of Week 3; the back half improves and proves it rather than adding architecture.

### Week 1 — Foundations & domain commitment

*Decide what we are proving, on what tasks, measured how. This week de-risks everything downstream.*

- Choose the **task basket**: 4–6 consulting task types (e.g. document summarization, contract/term extraction, document Q&A over a corpus, house-style memo drafting). Mark which 2–3 get specialist adapters.
- Confirm **labelability**: the team must be able to reliably score good vs. bad on every chosen task. If not, swap the task. This is the single biggest risk.
- Build the **benchmark v0**: 15–25 tasks per type with reference inputs; agree the scoring rubric (relevance, accuracy, currency, hallucination, 1–5).
- Stand up **local inference** on baremetal: base 70B serving + confirm multi-LoRA adapter swapping works end to end.
- Run the **frontier baseline** on benchmark v0 so there is a number to beat from day one.

**Exit criteria:** Task basket committed; benchmark v0 + rubric live; frontier baseline scored; base model serving with hot-swap adapters demonstrated.

## Week 2 — Specialists & the thin generalist

*Get both tiers minimally working in isolation. Bias toward a crude end-to-end over a polished half.*

- Train **2–3 specialist adapters** in parallel (Colab) for the chosen recurring task types; register them in the adapter registry.
- Stand up the **ensemble generalist v0**: 2–3 models generating candidates + a simple selector (LLM-as-judge or best-of-N) — judge training comes later.
- Score specialists and ensemble-v0 against the frontier baseline on their relevant tasks.
- Run the **domain bake-off** (eval workbook): confirm which tasks the local approach can realistically match frontier on; adjust the basket if a task is hopeless.

**Exit criteria:** Specialist adapters beat the base model on their tasks; ensemble-v0 produces answers end to end; bake-off confirms a viable basket.

## Week 3 — Routing & first full slice

*Wire the tiers together. By end of week the whole system runs end to end and is measured — the critical milestone.*

- Build the **router v1**: classify task type, dispatch to specialist or escalate to generalist on low confidence. Start simple; log every routing decision.
- Integrate **router → specialist → generalist** into one request path returning a final answer.
- Run the **full routed system** across the entire task basket; score against frontier + the fine-tune-and-route baseline.
- Measure **misroutes** explicitly — they are the router's error signal for the back half.

**Exit criteria:** End-to-end routed system runs on the full basket and is scored against both baselines. This is the thin-slice gate; everything after is improvement.

## Week 4 — Train the judge; tighten routing

*Replace the crude selector with a real trained judge; cut the misroute rate. This is where quality climbs.*

- Generate **judge training data**: open-weight teacher labels candidate pairs at volume; the team **hand-corrects the hard cases** (the high-value labeling hours — staff them deliberately).
- Fine-tune the **judge adapter**; swap it into the generalist in place of the v0 selector; re-score.
- Improve the **router** from Week 3's misroute log: better features, tuned confidence threshold, escalation rules.

- Re-run the full benchmark; compare to Week 3 numbers to quantify the judge's and router's contribution.

**Exit criteria:** Trained judge beats the v0 selector on the benchmark; router misroute rate measurably down; full-system numbers improved over Week 3.

### Week 5 — Hardening, cost, and the long tail

*Make it robust and measure the economics. Stop adding architecture; make what exists trustworthy.*

- Stress the system on **novel / out-of-distribution** tasks to confirm graceful escalation to the generalist.
- Tune **sampling, candidate count, and budgets** for the quality/latency/cost tradeoff.
- Complete the **cost model**: cost per task and per *solved* task for the routed system vs. frontier vs. ensemble-on-everything (eval workbook).
- Fix the worst failure modes surfaced in scoring; re-label any disputed eval items for consistency.

**Exit criteria:** Cost-per-task curve complete; system degrades gracefully on novel tasks; failure modes triaged; benchmark scores stable across re-runs.

### Week 6 — Lock results & package the POC

*Freeze the system, run the final evaluation cleanly, and assemble the story. No new features.*

- **Feature freeze.** Run the final benchmark end to end on a clean configuration; record every KPI.
- Produce the **results write-up**: routed local system vs. frontier vs. fine-tune-and-route, on quality, latency, cost, and data-locality.
- Prepare a **live demo** path: a handful of representative tasks showing routing to specialist vs. generalist.
- Document **what worked, what didn't, and next steps** — including the deferred self-training-specialist loop as the headline future-work item.

**Exit criteria:** Final KPIs locked; results doc + demo ready; honest assessment of whether the system matches/approaches frontier on the basket, and at what cost.

## 5. Key risks and how the plan handles them

| Risk | Mitigation in the plan |
|---|---|
| Chosen tasks aren't labelable — can't tell good from bad, so nothing can be scored. | Week 1 makes labelability a gating decision; swap any task that fails it before building. |
| Router misroutes — sends hard tasks to narrow specialists that fail confidently. | Start with a confidence threshold that escalates aggressively to the generalist; log and tune misroutes in Weeks 3–4. |
| Over-building with Claude Code — lots of impressive infra, never rigorously measured. | Thin measured slice by end of Week 3; back half is improvement + proof, not new architecture. Eval is a staffed workstream. |
| Judge/teacher ceiling — judge can only approach its teacher's quality. | Open-weight teacher for volume + human-corrected hard cases to push past the teacher on the tasks that matter. |
| The simple fine-tune-and-route baseline beats the ensemble. | Treated as a legitimate outcome, not a failure: build that baseline explicitly and report honestly which approach wins. |

| Risk | Mitigation in the plan |
|---|---|
| Colab unreliability disrupts long-running components. | Colab only for batch adapter training; all serving/eval lives on baremetal. |

## 6. Definition of done

The six weeks succeed if, on the committed task basket and our own benchmark, we can show:

- A **fully local** routed system (router + specialist adapters + ensemble/judge fallback) running end to end on our hardware.
- Quality that **matches or approaches** the GPT/Claude baseline across the basket — with the honest per-task picture, not a single cherry-picked task.
- **Lower marginal cost** than running the ensemble on everything, and a clear cost-vs-frontier comparison.
- A credible comparison to the **fine-tune-and-route** baseline, whichever way it falls.

*A strong partial result — e.g. matching frontier on most of the basket and approaching it on the rest, cheaply and privately — is a successful POC. Broad open-ended parity is explicitly not the six-week bar.*